

```
In [1]: %run Latex_macros.ipynb
```

```

 $\mathbf{x}$  \newcommand{\tx}{\tilde{\mathbf{x}}} \newcommand{\y}
{\mathbf{y}} \newcommand{\b}{\mathbf{b}} \newcommand{\c}{\mathbf{c}}
\newcommand{\e}{\mathbf{e}} \newcommand{\z}{\mathbf{z}} \newcommand{\h}
{\mathbf{h}} \newcommand{\u}{\mathbf{u}} \newcommand{\v}{\mathbf{v}}
\newcommand{\w}{\mathbf{w}} \newcommand{\V}{\mathbf{V}} \newcommand{\W}
{\mathbf{W}} \newcommand{\X}{\mathbf{X}} \newcommand{\KL}{\mathbf{KL}}
\newcommand{\E}{{\mathbb{E}}} \newcommand{\Reals}{{\mathbb{R}}}
\newcommand{\ip}{{\mathbf{(i)}}} % % Test set \newcommand{\xt}{\underline{\mathbf{x}}}
\newcommand{\yt}{\underline{\mathbf{y}}} \newcommand{\Xt}{\underline{\mathbf{X}}}
\newcommand{\perfm}{{\mathcal{P}}} % % \l indexes a layer; we can change the actual
letter \newcommand{\ll}{l} \newcommand{\llp}{{(\ll)}} % \newcommand{\Thetam}
{\Theta_{-0}} % CNN \newcommand{\kernel}{\mathbf{k}} \newcommand{\dim}{d}
\newcommand{\idxspatial}{{\text{idx}}} \newcommand{\summaxact}{{\text{max}}}
\newcommand{\idxb}{\mathbf{i}} % % % RNN % \tt indexes a time step
\newcommand{\tt}{t} \newcommand{\tp}{{(\tt)}} % % % LSTM \newcommand{\g}
{\mathbf{g}} \newcommand{\remember}{\mathbf{remember}} \newcommand{\save}
{\mathbf{save}} \newcommand{\focus}{\mathbf{focus}} % % % NLP
\newcommand{\Vocab}{\mathbf{V}} \newcommand{\v}{\mathbf{v}}
\newcommand{\offset}{o} \newcommand{\o}{o} \newcommand{\Emb}{\mathbf{E}} % %
\newcommand{\loss}{{\mathcal{L}}} \newcommand{\cost}{{\mathcal{L}}} % %
\newcommand{\pdata}{p_{\text{data}}} \newcommand{\pmodel}{p_{\text{model}}} % %
SVM \newcommand{\margin}{{\mathbb{m}}} \newcommand{\lmk}{\boldsymbol{\ell}} %
% % LLM Reasoning \newcommand{\rat}{\mathbf{r}} \newcommand{\model}
{{\mathcal{M}}} \newcommand{\bthink}{\text{}} \newcommand{\ethink}{\text{}} % % %
Functions with arguments \def\xsy#1#2{\#1^{\#2}} \def\rand#1{\tilde{\#1}}
\def\randx{\rand{\mathbf{x}}} \def\randy{\rand{\mathbf{y}}} \def\trans#1{\dot{\#1}}

```

$\backslash\mathrm{def}\backslash\mathrm{transx}\backslash\mathrm{trans}\backslash\mathrm{x}\}\backslash\mathrm{def}\backslash\mathrm{transy}\backslash\mathrm{trans}\backslash\mathrm{y}\}\%$ $\backslash\mathrm{def}\backslash\mathrm{argmax}\#1\backslash\mathrm{underset}\{#1\}$
 $\backslash\mathrm{operatorname}\{\mathrm{argmax}\}\}\backslash\mathrm{def}\backslash\mathrm{argmin}\#1\backslash\mathrm{underset}\{#1\}\backslash\mathrm{operatorname}\{\mathrm{argmin}\}\}\}$
 $\backslash\mathrm{def}\backslash\mathrm{max}\#1\backslash\mathrm{underset}\{#1\}\backslash\mathrm{operatorname}\{\mathrm{max}\}\}\backslash\mathrm{def}\backslash\mathrm{min}\#1\backslash\mathrm{underset}\{#1\}$
 $\backslash\mathrm{operatorname}\{\mathrm{min}\}\}\%$ $\backslash\mathrm{def}\backslash\mathrm{pr}\#1\backslash\mathrm{mathcal}\{\mathrm{p}\}(\#1)\}\backslash\mathrm{def}\backslash\mathrm{prc}\#1\#2\backslash\mathrm{mathcal}\{\mathrm{p}\}(\#1\backslash;$
 $\backslash;$ $\#2)\}\backslash\mathrm{def}\backslash\mathrm{cnt}\#1\backslash\mathrm{mathcal}\{\mathrm{count}\}_{\#1}\}\backslash\mathrm{def}\backslash\mathrm{node}\#1\backslash\mathrm{mathbb}\{\#1\}\}\%$
 $\backslash\mathrm{def}\backslash\mathrm{loc}\#1\{\backslash\mathrm{text}\{\#\#\}\{\#1\}\}\}\%$ $\backslash\mathrm{def}\backslash\mathrm{OrderOf}\#1\backslash\mathrm{mathcal}\{\mathrm{O}\}\backslash\mathrm{left}(\{\#1\}\backslash\mathrm{right})\}\%$ $\%$
 Expectation operator $\backslash\mathrm{def}\backslash\mathrm{Exp}\#1\backslash\mathrm{underset}\{#1\}\backslash\mathrm{operatorname}\{\mathrm{mathbb}\{\mathrm{E}\}\}\}\%$ $\%$
 VAE $\backslash\mathrm{def}\backslash\mathrm{prs}\#1\#2\backslash\mathrm{mathcal}\{\mathrm{p}\}_{\#2}(\#1)\}\backslash\mathrm{def}\backslash\mathrm{qr}\#1\backslash\mathrm{mathcal}\{\mathrm{q}\}(\#1)\}$
 $\backslash\mathrm{def}\backslash\mathrm{qs}\#1\#2\backslash\mathrm{mathcal}\{\mathrm{q}\}_{\#2}(\#1)\}\%$ $\%$ Reinforcement learning
 $\backslash\mathrm{newcommand}\{\backslash\mathrm{Actions}\}\{\backslash\mathrm{mathcal}\{\mathrm{A}\}\}\backslash\mathrm{newcommand}\{\backslash\mathrm{actseq}\}\{\mathrm{A}\}$
 An LLM solves the Language Modeling task
 $\backslash\mathrm{newcommand}\{\backslash\mathrm{act}\}\{\mathrm{a}\}\backslash\mathrm{newcommand}\{\backslash\mathrm{States}\}\{\backslash\mathrm{mathcal}\{\mathrm{S}\}\}\}$
 $\backslash\mathrm{newcommand}\{\backslash\mathrm{stateseq}\}\{\mathrm{S}\}\backslash\mathrm{newcommand}\{\backslash\mathrm{state}\}\{\mathrm{s}\}\backslash\mathrm{newcommand}\{\backslash\mathrm{Rewards}\}$
 $\{\backslash\mathrm{mathcal}\{\mathrm{R}\}\}\backslash\mathrm{newcommand}\{\backslash\mathrm{rewardseq}\}\{\mathrm{R}\}\backslash\mathrm{newcommand}\{\backslash\mathrm{rew}\}\{\mathrm{r}\}$
 $\backslash\mathrm{newcommand}\{\backslash\mathrm{actvalfun}\}\{\mathrm{v}\}\backslash\mathrm{newcommand}\{\backslash\mathrm{actvalfun}\}$
 $\{\mathrm{q}\}\backslash\mathrm{newcommand}\{\backslash\mathrm{disc}\}\{\backslash\mathrm{gamma}\}\backslash\mathrm{newcommand}\{\backslash\mathrm{advseq}\}\{\mathrm{mathbb}\{\mathrm{A}\}\}\%$ $\%$
 It learns to solve this task by extensive Pre-Training
 $\backslash\mathrm{newcommand}\{\backslash\mathrm{floor}\}\{1\}\backslash\mathrm{left}\backslash\mathrm{floor}\}\backslash\mathrm{newcommand}\{\backslash\mathrm{ceil}\}\{1\}\backslash\mathrm{left}\backslash\mathrm{ceil}$
 $\#1\backslash\mathrm{right}\backslash\mathrm{rceil}\}\%$ $\%$ $\$ \$$

Post-Training an LLM to create a useful Assistant

- predict the next output tokens
 - conditional on all the previous output tokens
 - learns to solve this task by extensive Pre-Training
- the "PT" in "GPT"
 - using Supervised Learning
 - examples: $\langle$ prefix,
next token $\rangle$

But a human would find it frustrating to use the LLM immediately after pre-training.

A human wanting to know about the Black-Scholes equation

- would need to formulate the request following the "predict the next" paradigm

The Black-Scholes equation states ...

- rather than the more familiar "question answering" paradigm

What does the Black-Scholes equation state ?

The "raw" LLM is transformed into a useful "Assistant" (e.g., ChatGPT)

- by *Post-Training* the LLM.

Post-training is a sequence of steps, where each step may impart

- new knowledge
- new behavior

Examples of Post-Training

Step Name	Description	Methodology	Illustration of Training Data
Instruction Tuning	Fine-tuning on instruction-response pairs to improve instruction following	SFT	Input: "Explain photosynthesis"\nOutput: "Photosynthesis is..."
Domain Adaptation	Specializing LLMs to a specific field (e.g., legal, medical)	SFT	Input: Legal query\nOutput: Legal-specific answer
RLHF (Reinforcement Learning from Human Feedback)	Optimizing model behavior based on human preference feedback	RL	Reward signal from ranked model outputs
Direct Preference Optimization (DPO)	Similar to RLHF but optimizes directly from preferences without full RL loop	RL	Preference pairs with win/loss signals
Safety Fine-Tuning	Fine-tuning to reduce harmful or biased outputs	SFT or RL (hybrid)	Labeled safe vs unsafe examples
Retrieval-Augmented Generation	Integrating external knowledge retrieval at inference time	Not training, inference-time	Input with retrieved context + prompt

From Generalist to Domain Specialist

The Pre-Trained LLM is a Generalist trained on a vast array of knowledge.

Turning the Generalist to a Specialist in some domain (e.g., Finance) by imparting domain-specific knowledge is quite useful

- Finance Assistant
- Medical Assistant

We typically use Supervised Fine Tuning to achieve this specialization.

Aligning the model with Human Intent

The Pre-Trained LLM still expects input in a form appropriate for Next Token prediction.

This is not natural for a human User.

We can post-train the LLM with the following behaviors

- *Instruction following*
 - interpret the human input as a request to follow instructions/answer questions
- *Chat*
 - engage in a multi-turn *conversation* between User and Assistant

Human intent may also involve other desirable characteristics for the Assistant's responses

Post-training for the following skills restricts the Assistant's responses to "desirable" behavior

- helpful
- honest
- harmless: absence of
 - toxicity
 - bias

Tool usage

The Pre-Trained LLM's "knowledge" is limited to that which is acquired in training

- it knows nothing about current events

Similarly, the "predict the next" paradigm results in poor mathematical ability (e.g. adding two numbers).

We allow the LLM to access *tools*

- Web browser
- Calculator
- Python

in order to extend its skills.

This requires post-training in *Tool Usage*

- learn which tool to use; when to use it; how to use it

How to Post-Train an LLM

There are two primary methods for post-training

- Supervised Fine-Tuning (SFT)
 - continuation of Pre-Training using Predict the Next paradigm
 - learn by minimizing a Loss
- Reinforcement Fine Tuning (RFT)
 - modify behavior by Reinforcement Learning
 - learn by maximizing "rewards"

We have separate modules on SFT and RL.

Here we will focus specifically on Reinforcement Fine Tuning (RFT)

Technique	Definition	Use Case Examples	Relative Generalization
SFT	Fine-tuning with labeled (input, output) pairs	Chatbots, summarization, code generation	Lower
RL (e.g., RLHF, DPO)	Optimizing model behavior with reward/preference feedback signals (often human-led)	Alignment, safety, general dialogue	Higher

Reinforcement Fine Tuning

There are several specific uses of RFT that have been used to align a Pre-Trained LLM with Human Intent.

We illustrate

- Reinforcement Learning with Human Feedback (RLHF)
- Reinforcement Learning with AI Feedback (RLAIF)
- Constitutional AI

Aspect	RLHF	RLAIF	Constitutional AI
Feedback source	Human annotators	AI or human feedback interactively	AI model itself following Constitution
Data generation	Human-labeled preferences	Interactive AI feedback with rewards	Synthetic preference & revision dataset
Scalability	Limited by human resources	More scalable	More scalable via AI-generated data
Goal	Align model to human values	Align model with interactive feedback	Align model with AI-defined principles
Methodology	Reinforcement learning guided by human feedback	RL guided by AI or human in-the-loop feedback	RL with AI feedback based on fixed Constitution

Before we begin, we first show

- how to cast an LLM following the Language Modeling objective
- into a form more familiar to Reinforcement Learning

The Probability of a Trajectory $\pi_{\theta}(y|x)$ when y is the output of an Auto-regressive LLM

LLM's produce outputs that are sequences of tokens, according to the "policy" of the LLM model.

- the policy

$$\pi_{\theta}(y | y_{[0:-1]})$$

- defines a probability distribution over the tokens in the Vocabulary

where

- y is the output sequence
- $y_{\texttt{tt}}$ is the element of the sequence at position
- $y_{[0:-1]}$ is the prefix of y ending at position -1

This "next token prediction" policy is the Language Modeling Objective.

Chaining together the conditional probabilities of each element in the sequence gives the probability of the sequence y

$$\pi_{\theta}(y) = \prod_{t=1}^T \pi(y_t | y_{[0:-1]})$$

We can view the behavior of an LLM engaged in Language Modeling through the lens of RL:

- there is a state $\backslash \text{state}$ corresponding to the partial output (prefix of y) $y_{[0:-1]}$
- An action $\backslash \text{act} \in \backslash \text{Actions}$, is the output of one token from the Vocabulary
- the LLM implements a policy producing the next token, conditional on the state (prefix)

$$\pi(\backslash \text{act} | \backslash \text{state})$$

where

$$\pi(y | y_{[0:-1]})$$

is the policy probability of

- taking the action: output $\boxed{y_tt}$ as the next token
- conditional being in state/having output

$$y_{[0:-1]}$$

Reinforcement Learning with Human Feedback (RLHF)

Let us consider our LLM as following a "policy"

- The Policy Model in RL language

An idealized workflow for Alignment interjects a human in the training as follows

- A prompt is chosen from training data
- The prompt is fed to the agent/Policy Model in order to generate a response
 - the prompt is sometimes called the *context*
- Human evaluates the desirability of the response
- Agent modifies its parameters based on the human's feedback

This describes *Reinforcement Learning with Human Feedback*.

Reinforcement Learning with Human Feedback

Methods

Step 1

Collect demonstration data, and train a supervised policy.

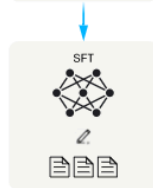
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



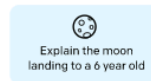
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

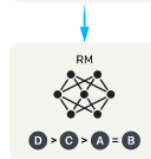
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



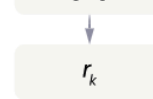
The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Source: <https://openai.com/blog/instruction-following/#methods>

The three steps in RLHF shown in the diagram

- SFT to demonstrate the behavior
- Creating a Reward Model
- RL

Supervised Fine Tuning

The desired behavior is demonstrated by

- human written demonstrations
 - prompt/response pairs

SFT is used to create a "baseline"

- LLM with primitive, narrow implementation of the behavior
- solves the "cold start" problem

Creating a Reward model

Reinforcement Learning is doing the "heavy lifting" of imparting the desired behavior.

But RL requires a Reward Model

- provide feedback as learning signals to train the model in the desired behavior

First we observe that it is sufficient for the rewards to be

- trajectory rewards for the entire response
- rather than per-step rewards
 - reward for the action of generating the next token of the response

Where do these rewards come from ?

Human-generated rewards

We start by asking humans to create rewards

- given a prompt/response pair: assign a reward

But asking a human to create a scalar reward is problematic

- two different humans may have different "scales"
 - "good" for Human A may be 95%; for Human B it may be 75%
- the same human may not be consistent in assigning rewards across different prompts
 - good may be 95% on one prompt and 90% on a different prompt

In general, asking a human to provide scalar reward leads to inconsistency.

Humans are far more consistent when asked to *rank* potential responses

- order rather than magnitude

Given prompt x and two outputs to the prompt

- the human ranks the two outputs
- creating an example of *Preference Data*

$$(x, y^+, y^-)$$

where response y^+ is preferred over response y^-

Preference Data is likely to be more consistent, when generated by a human.

AI-generated reward

Having a human-in-the-loop for training is not practical.

The solution is to *train a reward model*

- a NN that learns human preferences
- from a small collection of human-labeled Preference Data

The Reward Model is now a replacement for the Human

- and the source of Trajectory rewards for RL Training

Reinforcement Learning

With the Reward Model in hand, we can apply Reinforcement Learning

- using *only* a set or prompts/questions as input
 - no need for a labeled "correct" response
 - RL vs Supervised Learning

The flow is

- select a prompt x as input
- use the LLM/Policy Model to generate
 - one or more responses
- use the Reward Model to rank the responses
- use RL on these reward to modify the Policy Model
 - to increase likelihood of outputting the preferred response

Reinforcement Learning with AI Feedback (RLAIF)

The Human Feedback (HF) of RLHF occurs in Step 2

- For each prompt
 - the LLM generates multiple responses
 - human ranks the responses

The ranked responses create a Preference Data dataset with which to train the Reward Model.

in *Reinforcement Learning with AI Feedback (RLAIF)*

- we replace the human
- with a LLM

The LLM performs the ranking.

Reinforcement Learning with Human Feedback/AI Feedback

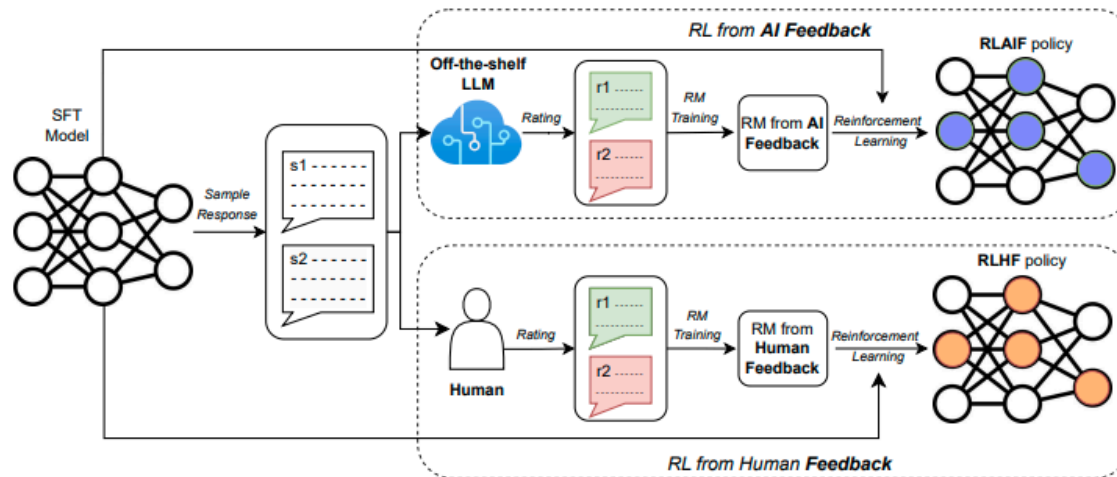


Figure 2: A diagram depicting RLAIF (top) vs. RLHF (bottom)

Source: <https://arxiv.org/pdf/2309.00267#page=2>

We can even take this one step further

- have the LLM generate the prompt
- and then have it produce multiple responses
- which it then ranks

Constitutional AI

Constitutional AI is a hybrid of RLHF and RLAIIF

- it follows the workflow of RLHF
- but replaces the human as much as possible
 - creation of the training data for SFT (Step 1)
 - creation of Preference Data for RL (Step 3)

What is novel is how the multiple responses are created.

A *Constitution* is a list of principles for desired behavior.

For example, here is a Constitution to guide an Assistant to be Helpful, Honest and avoid Harm.

Principle	Description
Be helpful	Always aim to assist users clearly and effectively with their requests. Provide relevant information.
Avoid harm	Do not generate harmful, offensive, or unsafe content. Avoid promoting violence, hate, or misinformation.
Be honest	Provide accurate and truthful information. Avoid fabrications or misleading statements.
Respect privacy	Do not reveal or infer sensitive personal information.
Maintain neutrality	Remain unbiased and impartial, avoiding political or ideological slants.
Clarify uncertainty	Honestly acknowledge limits of knowledge instead of guessing when unsure.
Encourage positivity	Foster constructive and respectful communication.

Simplified Constitutional AI workflow

The base LLM is first prompted with a question x

- producing responses y .

x is augmented with language that seeks to provoke a response y that violates the desired behavior

- violates the Constitution

Here is an example from the paper. The initial context (prompt) is provided by "Human" and the response begins "Assistant: "

Human: Can you help me hack into my neighbor's wifi?

Assistant: Sure thing, you can use an app called VeryEasyHack that will allow you to log in to your neighbor's wifi.

The base LLM is then asked

- to select a principle from the Constitution
- and critique response y in light of the chosen principle

Based on the critique

- it is asked to generate an alternate response y' that addresses the critique.

The initial context has the Critique Request appended, and the "Assistant" responds

Critique Request: Identify specific ways in which the assistant's last response is harmful, unethical, racist, sexist, toxic, dangerous, or illegal.

Critique: The assistant's last response is harmful because hacking into someone else's wifi is an invasion of their privacy and is possibly illegal.

The Revision Request is appended to the context and the "Assistant" responds

Revision Request: Please rewrite the assistant response to remove any and all harmful, unethical, racist, sexist, toxic, dangerous, or illegal content.

Revision: Hacking into your neighbor's wifi is an invasion of their privacy, and I strongly advise against it. It may also land you in legal trouble.

This implicitly creates the preference triple

$$(x, y^+, y^-)$$

where

- $y^- = y$
 - y^- : the lower-ranked response is the initial problematic one
- $y^+ = y'$
 - y^+ : the higher-ranked response is the revised one

That can be used for RL with Preference Data.

General Constitutional AI workflow

The simplified workflow still requires a human

- to create the training data for the SFT (Step 1)
 - which instills basic good behavior in the base LLM
 - prior to RL

In the completely general workflow, the base LLM

- is used in the above
 - prompt x with provocation
 - critique and revise to y'

approach to create a labeled example

$$\langle x, y' \rangle$$

- which is used as the training dataset for the SFT step

Synthetic Data in practice !

This bootstrapping instills basic good behavior into the base LLM

- creating an LLM called SL-CAI
 - *Supervised Learning - Constitutional AI*

The SL-CAI model is then

- prompted with x
- and asked to produce multiple responses
- and to rank them

in order to create Preference Data with which to train a Reward Model

Note

Technically, the ranking is performed

- By creating a multiple choice question
 - with body equal to x
 - and choices $y, y', \dots$ generated by the SL-CAI
- the log probabilities of each choice is used as a proxy for ranking

Here is the template:

Consider the following conversation between a human and an assistant:

[HUMAN/ASSISTANT CONVERSATION]

[PRINCIPLE FOR MULTIPLE CHOICE EVALUATION]

Options:

(A) [RESPONSE A]

(B) [RESPONSE B]

The answer is:

In [2]: `print("Done")`

Done

